Miłosz Kutyła

Application Security — laboratories
Topic #7: Security audit

March 17, 2026

Contents

1. Introduction . . . 1
2. Black box . . . 2
2.1. SQL injection tests . . . 2
2.2. Cross-Site Scripting (XSS) tests . . . 3
2.3. Testing the search functionality . . . 4
2.4. Session management tests . . . 4
2.5. RBAC tests . . . 4
2.6. Cross-Site Request Forgery (CSRF) tests . . . 4
2.7. Information disclosure via registration form . . . 4
2.8. Tinkering with request parameters . . . 5
2.9. Trying to access removed assets . . . 6
2.10. Issues with deleted accounts . . . 7
2.11. Automated scan . . . 9
3. White box . . . 10
3.1. Positive findings . . . 10
3.1.1. SQL injection mitigation . . . 10
3.1.2. XSS mitigation . . . 10
3.1.3. Secure randomness usage . . . 10
3.1.4. Role-Based Access Control (RBAC) implementation . . . 10
3.2. Negative findings . . . 11
3.2.1. Information disclosure through registration endpoint . . . 11
3.2.2. Hard deletion of soft-deleted accounts . . . 11
3.2.3. Transport layer security gaps . . . 12
3.2.4. Image validation limitation . . . 13
3.2.5. Input validation gaps . . . 13
4. Conclusions . . . 14

1. Introduction

The objective of this laboratory exercise was to conduct a security assessment of the web application developed by the group "AS_1_G" (source: `https://github.com/kamo104/app-sec`). This assessment employed a two-step approach: black box testing (focusing on finding weaknesses leading to unauthorized access, data exposure, or system malfunction using both manual and automated verification techniques) followed by white box analysis to inspect the source code for the identification of logic flaws, insecure design patterns or to verify findings and determine root causes.

2. Black box

I verified the application's resilience against common web vulnerabilities, focusing on SQL injection, Cross-Site Scripting (XSS), malicious file uploads, and logic flaws.

2.1. SQL injection tests

To test for SQL injections, I injected standard SQL payloads like `' OR 1=1 --` into all forms (login, registration, forgotten password, search). The application consistently returned generic error messages or treated the input as literal text. My attempts of injecting the search bar is shown in Figure 1.

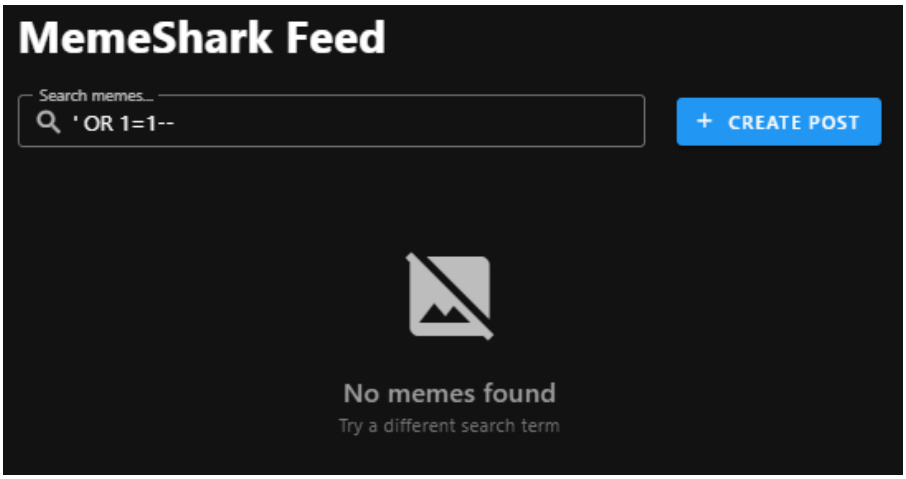


Figure 1: Manual attempt of SQL injection

I observed that searching for the specific wildcard character `%` returns all results, which initially raised suspicion of improper input handling. However, further testing confirmed that this is intended behavior for the search functionality (likely passing the input to a `LIKE` pattern) and is not exploitable for SQL injection. I also check for the `?` and `*` characters, but neither worked. An example of querying for posts matching pattern `%t%` is shown in Figure 2.

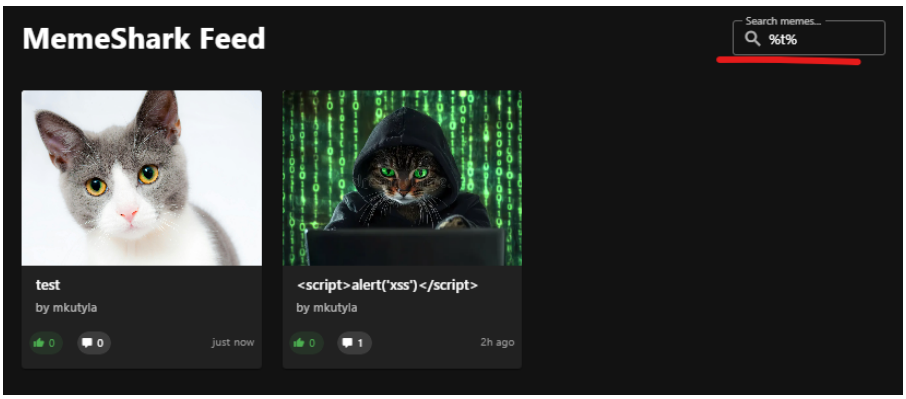


Figure 2: Wildcard functionality in search bar

I additionally performed automated scans using `sqlmap` on all the above forms. The tool did not identify any injection points, indicating effective input sanitization or the use of parameterized queries throughout the application. An example of using sqlmap for attacking the login and search forms is shown in Figure 3.

```
┌──(kali㉿kali)-[~]
└─$ sqlmap -u "https://192.168.1.7:4000/api/login" \
  --data='{"username":"*","password":"*"}' \
  --headers="Content-Type: application/json" \
  --ignore-code=401 \
  --batch --level=3 --risk=2
```

```
[13:53:17] [WARNING] (custom) POST parameter 'JSON #2*' does not seem to be injectable
[13:53:17] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk'
 options if you wish to perform more tests. If you suspect that there is some kind of protection mechanism involved (e.g.
 WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or switch '--random-agent'
```

(a) Testing the login form

```
┌──(kali㉿kali)-[~]
└─$ sqlmap -u "https://192.168.1.7:4000/api/posts/search?q=*&limit=*&offset=*" --level=3 --risk=2 --batch
```

```
[13:34:42] [WARNING] URI parameter '#3*' does not seem to be injectable
[13:34:42] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk'
 options if you wish to perform more tests. If you suspect that there is some kind of protection mechanism involved (e.g.
 WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or switch '--random-agent'
[13:34:42] [WARNING] HTTP error codes detected during run:
```

(b) Testing the search functionality

Figure 3: Using sqlmap to look for SQL injection

2.2. Cross-Site Scripting (XSS) tests

I tested the system for stored and reflected XSS:

- stored: I attempted to inject scripts into post titles, descriptions, comments, and username (a specific example, as the app allows usernames that have 20 characters or less).
- reflected: I injected payloads into the search parameter.

The application correctly escaped all malicious payloads, rendering them as harmless text. This suggests the frontend handles context-aware auto-escaping effectively. Escaped results are shown in Figure 4 and 5.

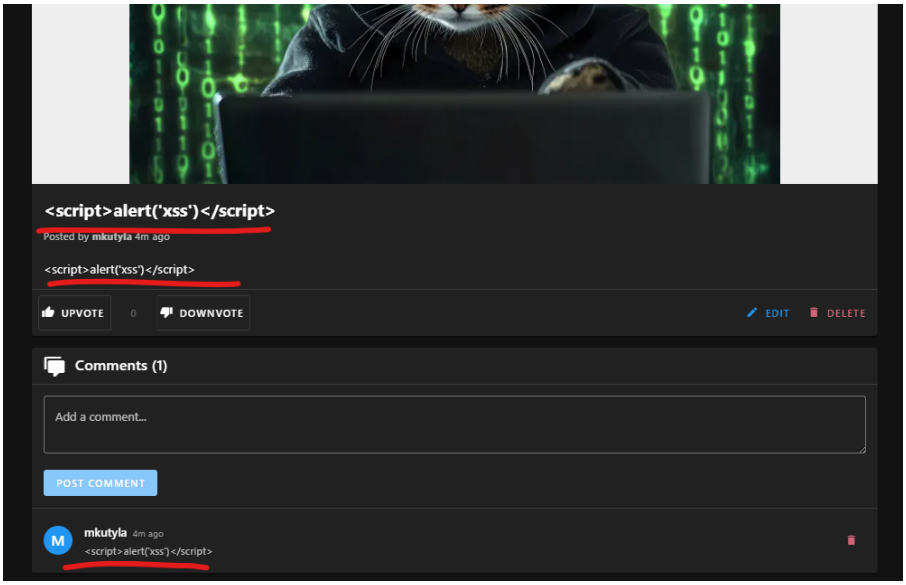


Figure 4: XSS attempts in: post title, content, and comments

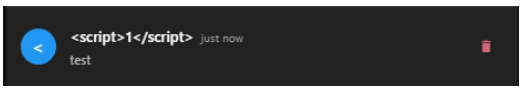


Figure 5: Failed XSS attempts: post title, content, comments, and username

2.3. Testing the search functionality

I examined the search functionality for logic flaws but didn't find anything relevant:

- I verified that searching for unique strings contained in removed posts did not return those posts to users.
- as previously mentioned, I discovered that the search function supports the SQL wildcard `%` character. Searching for just `%` returns all posts. While not a vulnerability per se, it indicates the input is passed to a `LIKE` clause.

2.4. Session management tests

I examined the configuration of session cookies to verify their resilience against client-side attacks. I observed that the application enforces the `HttpOnly` flag on the `session` cookie, rendering it inaccessible to JavaScript and effectively preventing session theft via XSS. Additionally, the cookie is configured with the `SameSite=Strict` attribute. This setting ensures the browser does not send the cookie with cross-site requests, providing robust protection against Cross-Site Request Forgery (CSRF) attempts. The cookie along with its configuration is shown in Figure 6.

Name	Value	Domain	Path	Expir...	Size	Http...	Secure	Sam...	Parti...	Cros...	Priori...
session_token	be3db65f455a969a3d9c409c4d...	localhost	/	2026...	77	✓	✓	Strict			Medi...

Figure 6: Session cookie

2.5. RBAC tests

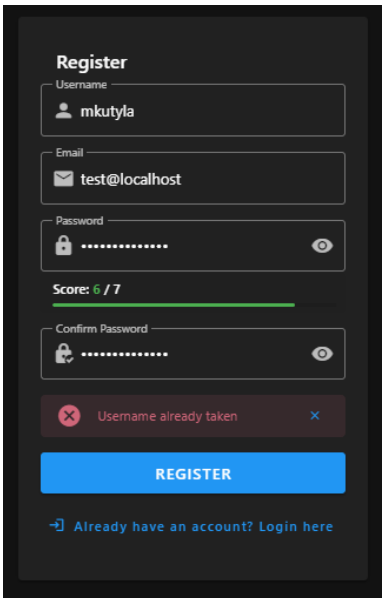
I examined the application's Role-Based Access Control (RBAC) and privilege separation mechanisms. I attempted to interact with protected endpoints without a valid session cookie. Then I tried accessing `/api/admin/*` endpoints without a session and with a standard user session. The site correctly returned "403 Forbidden" or "401 Unauthorized" in both cases. My tests indicate that privilege boundaries are correctly enforced.

2.6. Cross-Site Request Forgery (CSRF) tests

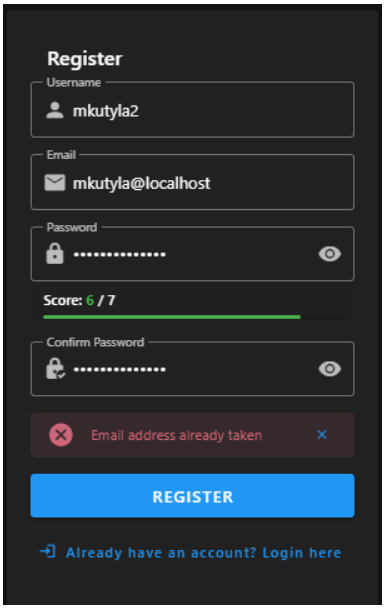
I analyzed the application's protection against Cross-Site Request Forgery (CSRF). The application does not require a unique anti-CSRF token for sensitive state-changing actions such as deleting posts or banning users, which would normally indicate a potential CSRF vulnerability. To assess exploitability, I created a PoC HTML page containing a hidden form that issued a cross-origin request to the post deletion endpoint. However, the attack was unsuccessful. Although the application logic lacks explicit anti-CSRF tokens, the session cookie is configured with the `SameSite=Strict` attribute. This configuration prevents the browser from including the session cookie in cross-origin requests. This mitigates the CSRF attack by browser-enforced cookie policy rather than application-level validation.

2.7. Information disclosure via registration form

I examined the login, register, and forgotten password forms to look for any potential misconfiguration. I noticed that after trying to register an account with username or email that's already registered, I receive a message saying it's taken, as shown in Figure 7. As emails are not public, this is information disclosure.



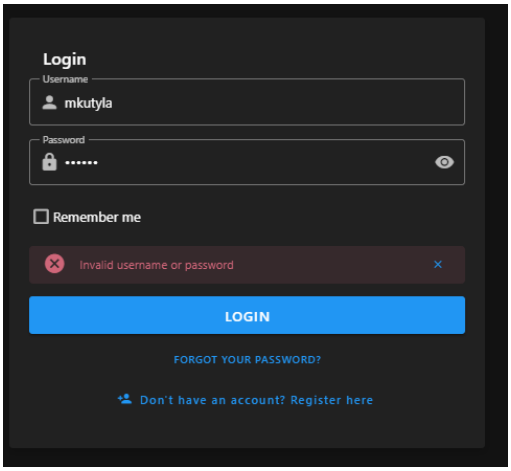
(a) Username taken (public information)



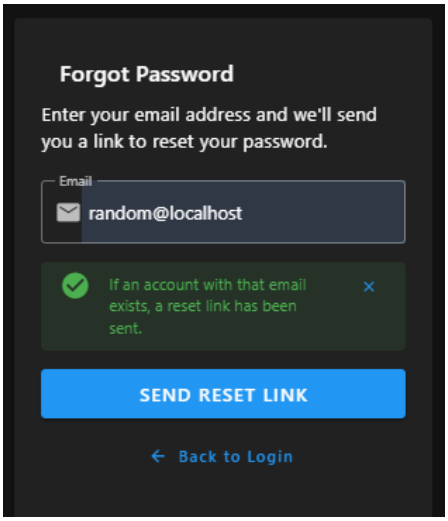
(b) Username taken (non-public information)

Figure 7: Information disclosure through register form

The login and forgot password forms return ambiguous error messages making them secure against enumeration attacks, as shown in Figure 8.



(a) Login form



(b) Forgotten password form

Figure 8: No information disclosure through login and forgotten password form

2.8. Tinkering with request parameters

I attempted to bypass business logic constraints by manually manipulating request parameters intercepted with BrupSuite but didn't manage to find anything. For example, I targeted the post rating mechanism, which expects a value of "1" (upvote) or "-1" (downvote). I modified the request to send arbitrary integer values such as "100" and "-2", attempting to artificially inflate a post's score. The server correctly identified these invalid values and responded with "400 Bad Request", as shown in Figure 9. This confirms that the backend strictly enforces the allowed enum values for ratings and does not blindly trust client input.

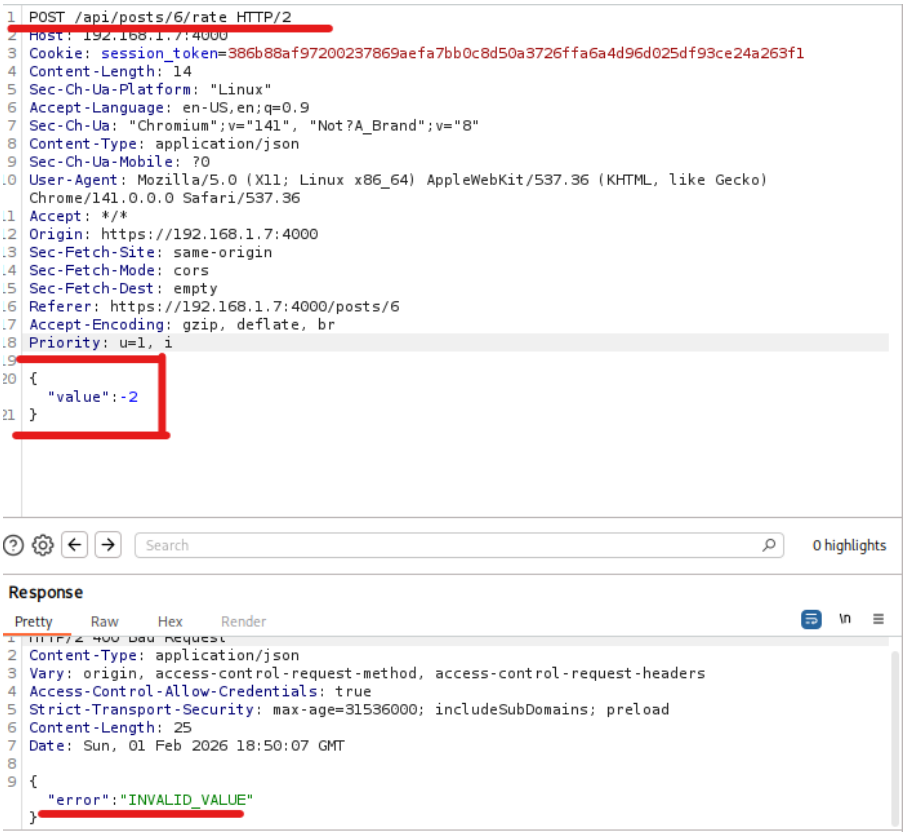


Figure 9: Tinkering with rating values

2.9. Trying to access removed assets

I tested the access control enforcement on uploaded media files by uploading a post with an image and noting their URLs. After removing the post, I attempted to access them both using the direct URLs. The post got properly removed along with the embedded image, as shown in Figure 10. The same is true for post's comments.

(a) Image found before post removal

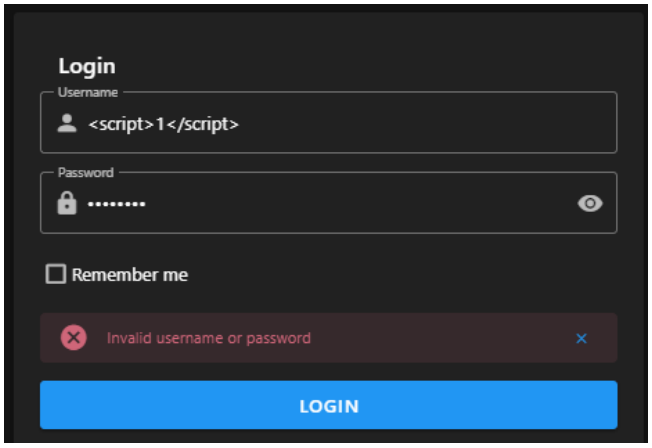
(b) Image not found after removal

(c) Post not found after removal

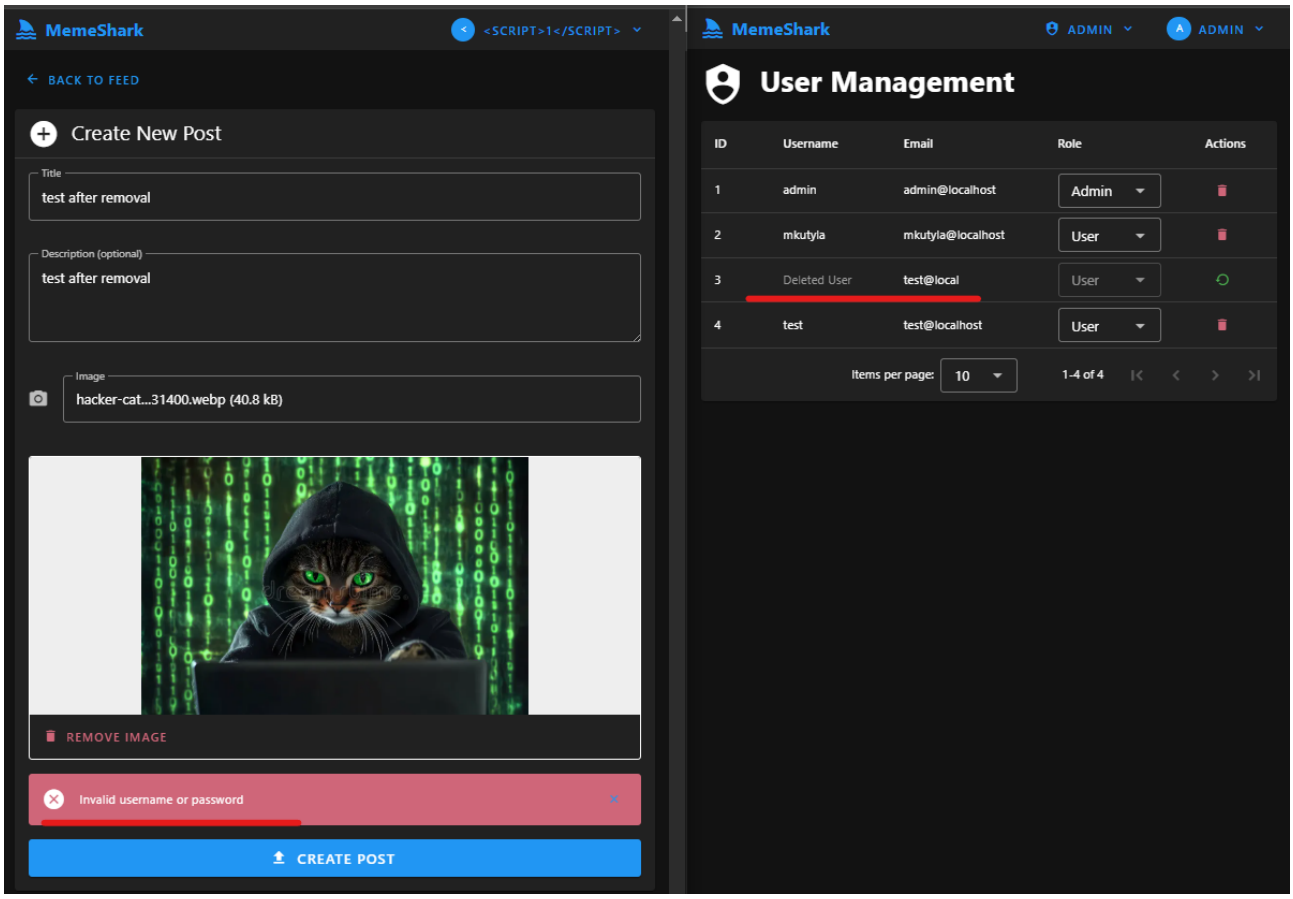
Figure 10: Trying to access removed assets

2.10. Issues with deleted accounts

I also tested the account deletion process to check access termination. As illustrated in Figure 11a, the account is immediately locked out, preventing any subsequent login attempts. Furthermore, I verified that any active sessions are correctly invalidated on the server-side. Although the session cookie may persist on the client-side, it is rendered non-functional, causing any subsequent requests to fail as shown in Figure 11b, effectively denying access to the deleted account.



(a) Account locked out, can't log in

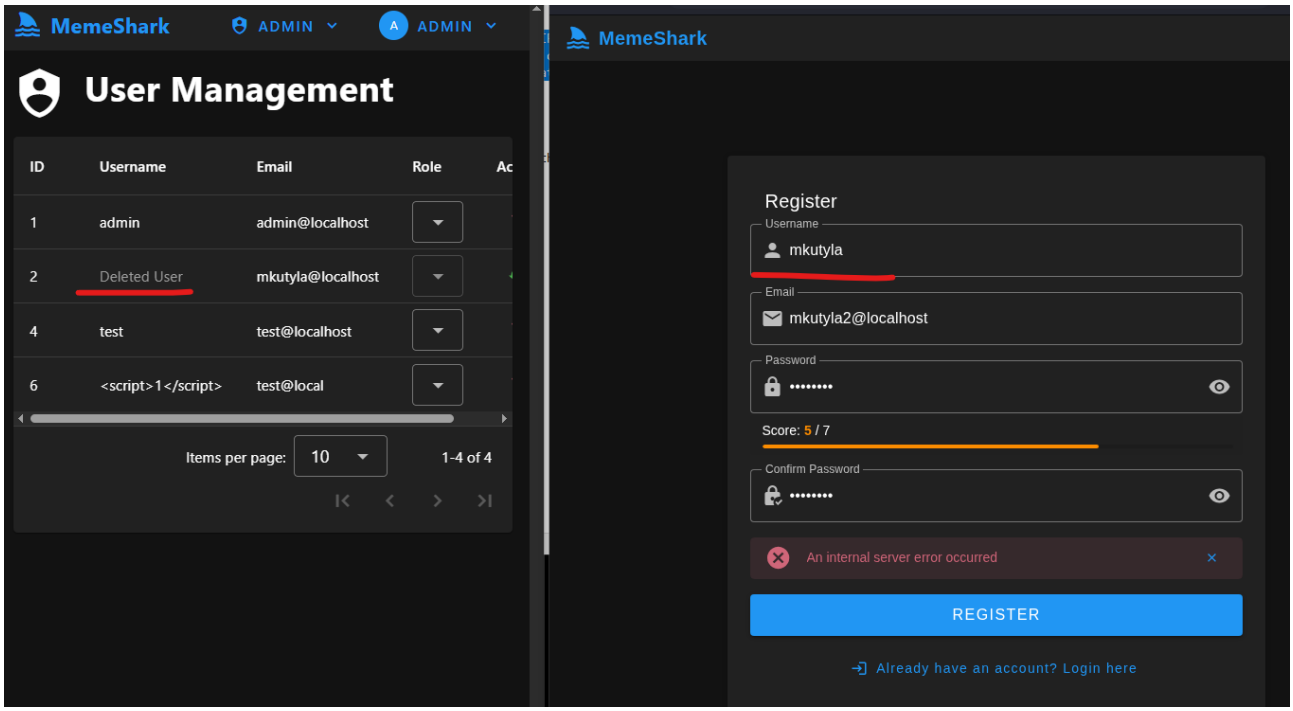


(b) Session is not disposed client-side but is nonfunctional

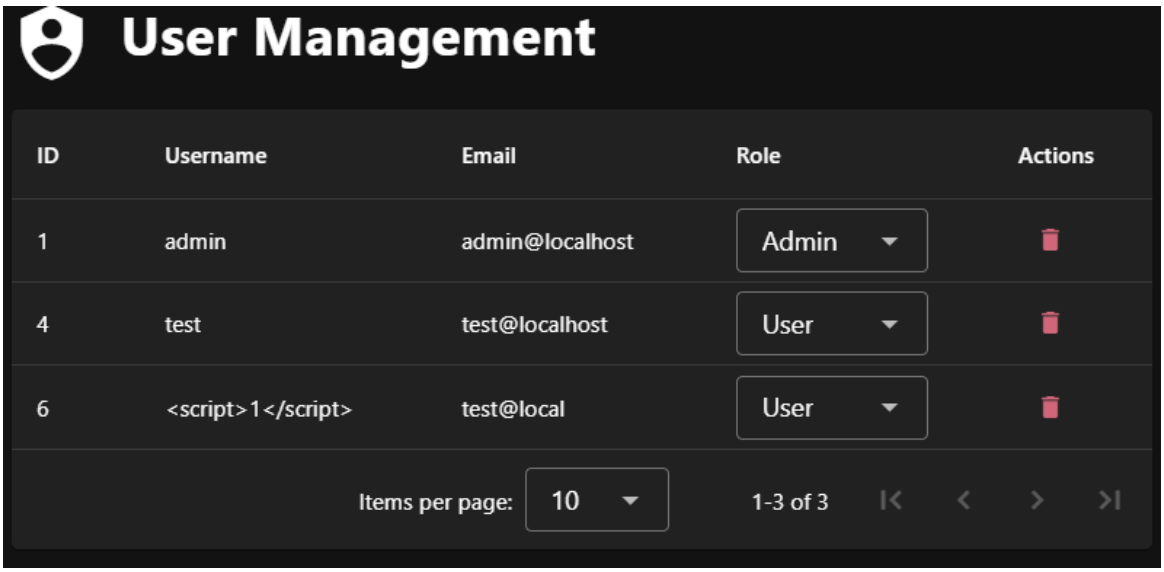
Figure 11: Actions after account removal

I also attempted to register a new account using a username and email address that I knew were already present in the database but were deleted by an admin. I specifically targeted the soft-deleted account with username “mkutyla”:

1. I attempted to register a second time with the same username “mkutyla”. The server returned a specific error ”An internal server error occurred”. As a result the soft-deleted account got hard-deleted, as shown in Figure 12.



(a) Registering



(b) The soft-deleted record gets hard-deleted

Figure 12: Registering with a username of a deleted account

After that, I tried to resend the form. A new account got created, as shown in Figure 13.

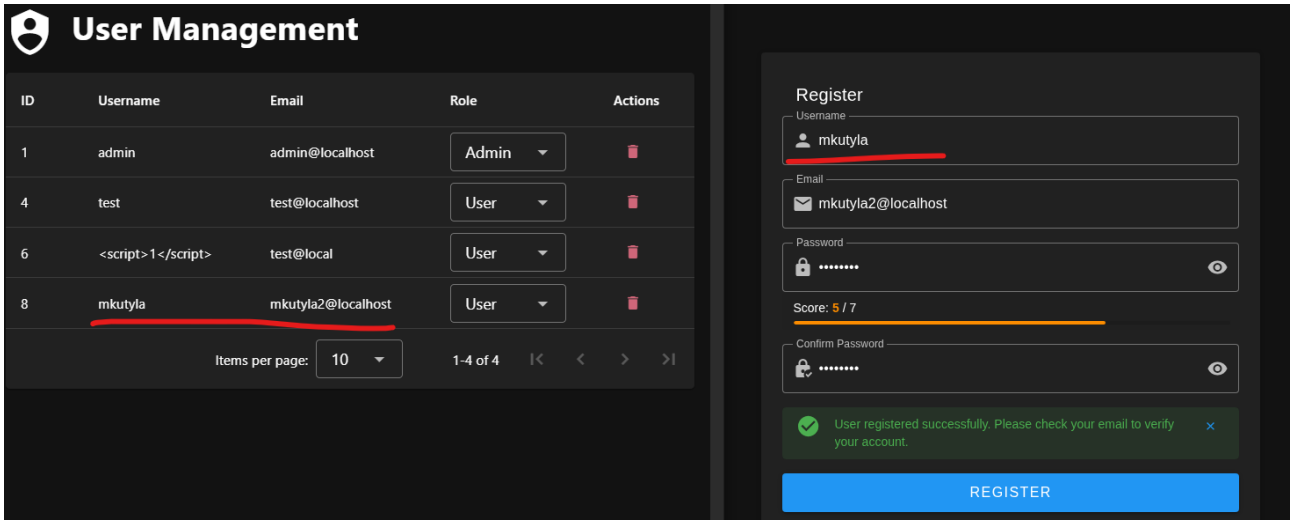


Figure 13: Creating a new account with the re-used username

2. The same was possible when re-using the email of a soft-deleted account, although given the fact that emails are non-public, it is a weaker attack vector.

This distinct error allows an attacker to effectively remove soft-deleted users, knowing either their username (which was public at some point) or email.

2.11. Automated scan

I performed an automated scan with OWASP ZAP. It yielded mediocre results and didn't find any major issues with the app. It raised alerts due to lack of three headers: CSP, anti-clickjacking and X-Content-Type-Options. Additionally, it highlighted timestamps being returned as part of posts. While these are not critical vulnerabilities, the missing headers are important security measures:

- Content Security Policy (CSP): mitigates XSS attacks by restricting the sources from which the browser can load resources.
- X-Frame-Options: prevents clickjacking by disallowing the site from being rendered within an iframe.
- X-Content-Type-Options: prevents "MIME-sniffing," ensuring browsers respect the declared content type and do not accidentally execute non-script files as JavaScript.

ZAP also raised some informational alerts:

- comments present in some `.js` files: nothing of importance,
- tagged the app as being modern,
- lack of cache-control header allowing the browser and proxies to cache content.

The scan results are presented in Figure 14.

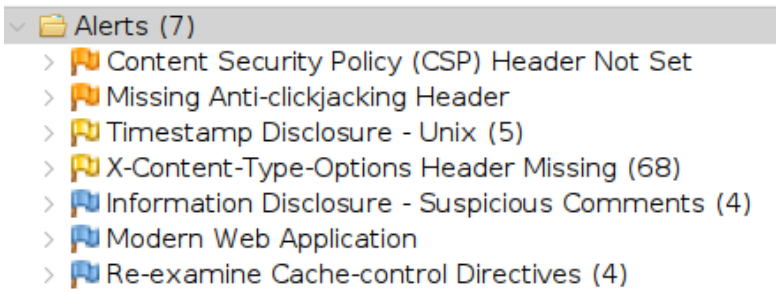


Figure 14: OWASP ZAP scan results

3. White box

3.1. Positive findings

3.1.1. SQL injection mitigation

I inspected the database interaction layer implemented in rust files located in `backend/src/db/`. My inspection confirmed that the application uses a library which enforces parameterized queries for all database interactions (`sqlx`). This approach ensures that user input is treated strictly as data, effectively neutralizing SQL injection attacks. An example of this is shown in Figure 15.

```
// backend/src/db/user_login.rs
pub async fn get_by_username(&self, username: &str) -> \
 std::result::Result<UserLogin, sqlx::Error> {
    let out = sqlx::query_as::<_, UserLogin>(formatcp!(
        "SELECT␣*␣FROM␣{}␣WHERE␣username␣=␣$1␣AND␣deleted_at␣IS␣NULL",
        UserLoginTable::TABLE_NAME
    ))
    .bind(username) // Parameter binding
    .fetch_one(&self.conn_pool)
    .await?;
    Ok(out)
}
```

Figure 15: Parameterized query usage in user retrieval

3.1.2. XSS mitigation

The application frontend is built with Vue.js, which provides built-in context-aware auto-escaping. All data binding uses the standard mustaches syntax {{ }}, which automatically escapes HTML entities. An example of this is shown in Figure 16.

```
<!-- frontend/src/pages/posts/[id].vue -->
<h1 class="text-h4␣font-weight-bold">{{ post.title }}</h1>
<p class="text-body-1␣mt-4" style="white-space:␣pre-wrap;">{{ post.description }}</p>
```

Figure 16: Safe data binding in Vue templates

3.1.3. Secure randomness usage

I verified the cryptographic primitives used for token generation in `backend/src/db/mod.rs`. The application correctly utilizes `rand_core::OsRng`, which interfaces with the operating system's CSPRNG (Cryptographically Secure Pseudo-Random Number Generator). This ensures that session tokens, password salts, and verification codes are unpredictable and resistant to entropy exhaustion attacks.

3.1.4. Role-Based Access Control (RBAC) implementation

I analyzed the authorization logic in `backend/src/api/auth_extractor.rs` and `backend/src/api/admin.rs`. The application implements a type-safe extractor pattern for protecting administrative endpoints. The `AdminUser` extractor wraps the "AuthenticatedUser" extractor and performs an explicit check against the `UserRole::Admin` enum before allowing the request to proceed. This ensures that improved privileges are checked before the handler logic is executed. This is shown in Figure 17.

```rust
// backend/src/api/auth_extractor.rs
impl<S> FromRequestParts<S> for AdminUser {
    async fn from_request_parts(parts: &mut Parts, state: &S) -> \
    Result<Self, Self::Rejection> {
        // ... authentication checks ...
        if auth_user.user.role != UserRole::Admin {
            return Err(AdminRejection::Forbidden);
        }
        Ok(Self(auth_user))
    }
}
```

Figure 17: AdminUser extractor enforcing RBAC

3.2. Negative findings

3.2.1. Information disclosure through registration endpoint

In the registration flow, the backend returns distinct error codes (`UsernameTaken`, `EmailTaken`) when a user attempts to register with existing credentials, leading to information disclosure (in case of non-public emails). This is shown in Figure 18.

```rust
// backend/src/api/register.rs
match db.user_login_table.is_email_free(&payload.email).await {
        Ok(true) => {
            debug!("Email␣'{}'␣is␣available", payload.email);
        }
        Ok(false) => {
            debug!(
                "Registration␣failed:␣email␣'{}'␣already␣taken",
                payload.email
            );
            return (
                StatusCode::CONFLICT,
                Json(RegisterErrorResponse {
                    error: RegisterError::EmailTaken,
                    validation: None,
                }),
            )
                .into_response();
}
```

Figure 18: Explicit error code confirming username existence

3.2.2. Hard deletion of soft-deleted accounts

As mentioned in the black-box part, I identified a critical logic flaw in the registration "cleanup" mechanism that leads to the permanent loss of data. When a user registers, the system checks if the username is free using code that explicitly ignores soft-deleted users (`deleted_at IS NULL`). What happens is:

1. A registration attempt for a soft-deleted username passes the initial application check.
2. The `INSERT` operation fails at the database level due to the `UNIQUE` constraint.
3. The application catches this error and triggers a "cleanup" routine, intended to delete the partially created user.
4. The cleanup routine calls `delete(username)`, which performs a hard delete (`DELETE FROM ...`) based on the username. Since the new user was never created, this operation mistakenly targets and permanently destroys the existing soft-deleted user record.

   The root cause for this issue is shown in Figure 19

```rust
// backend/src/api/register.rs
//
// v--- Checking if username exists: passes
match db
    .user_login_table
    .is_username_free(&payload.username)
    .await
{
    Ok(true) => {
        debug!("Username ’{}’ is available", payload.username);
    }

// backend/src/api/user_login.rs
// v--- called by the registration check
pub async fn is_username_free(&self, username: &str) -> Result<bool> {
        return match self.get_by_username(username).await
        ...

// v--- called by is_username_free, returns nothing as deleted_at is not NULL
pub async fn get_by_username(
    &self,
    username: &str,
) -> std::result::Result<UserLogin, sqlx::Error> {
    let out = sqlx::query_as::<_, UserLogin>(formatcp!(
        "SELECT * FROM {} WHERE username = $1 AND deleted_at IS NULL",
        UserLoginTable::TABLE_NAME
    ))

// backend/src/api/register.rs
//
// v--- this accidently removes the soft-deleted account
if let Err(e) = db.user_data_table.insert(user_id).await {
    // ...
    match db.user_login_table.delete(&payload.username).await {
        Ok(_) => debug!("Cleanup successful: deleted user ’{}’", payload.username),
        Err(cleanup_e) => error!(
            "Cleanup failed for user ’{}’: {:?}",
            payload.username, cleanup_e
        ),
    }
    return (
        StatusCode::INTERNAL_SERVER_ERROR,
        Json(RegisterErrorResponse {
            error: RegisterError::Internal,
            validation: None,
        }),
    )
        .into_response();
}
}
```

Figure 19: Registration cleanup logic removing soft-deleted user accounts

This vulnerability allows any attacker to permanently delete soft-deleted accounts and their associated data simply by attempting to register with the target’s username or email.

3.2.3. Transport layer security gaps

While analyzing the configuration and utility functions, I noted that the SMTP client implementation does not enforce TLS encryption. The current configuration in `backend/src/config.rs` and `backend/src/api/utils.rs`

sends sensitive email verification and password reset tokens over plaintext connections if the SMTP server allows it.

```
// backend/src/api/utils.rs
let mailer = AsyncSmtpTransport::<Tokio1Executor>::builder_dangerous(smtp_host)
    .port(smtp_port)
    .build();
```

Figure 20: Insecure SMTP transport configuration

3.2.4. Image validation limitation

I examined the file upload validation logic in `field-validator/src/lib.rs`. The current implementation relies solely on "Magic Byte" detection (inspecting the first few bytes of the file) to validate image uploads. It does not perform full image parsing or re-encoding. This allows for "Polyglot" files (that are valid images but also contain malicious code, e.g., PHP or HTML) to be uploaded and potentially served to users.

```
// field-validator/src/lib.rs
fn detect_image_mime_internal(data: &[u8]) -> Option<&'static str> {
    if data.starts_with(JPEG_MAGIC) { return Some("image/jpeg"); }
    // Only checks header bytes, not full content
    // ...
}
```

Figure 21: Shallow image validation

3.2.5. Input validation gaps

While checking files that perform any database calls, I identified missing input validation boundaries on the the `Pagination` struct in `api-types/src/requests.rs`. It allows arbitrary "limit" and "offset" values. This structure is used directly by the `list_posts` handler in `backend/src/api/posts.rs`, which passes these values to the database layer. In `backend/src/db/posts.rs`, these values are bound directly to the `LIMIT` and `OFFSET` clauses of the SQL query. This allows an attacker to request an extremely large limit, forcing the database to attempt to retrieve a massive dataset. If database was large enough, an attacker could request an extremely large limit to trigger database resource exhaustion.

```
// api-types/src/requests.rs
pub struct PaginationQuery {
    pub limit: i64, // No upper bound validation (min/max)
    pub offset: i64,
}

// backend/src/db/posts.rs
pub async fn get_all_visible(&self, limit: i64, offset: i64) -> Result<Vec<Post>> {
    sqlx::query_as(
        "SELECT␣*␣...␣LIMIT␣$1␣OFFSET␣$2" // Direct usage
    )
    .bind(limit)
    .bind(offset)
    .fetch_all(&self.conn_pool).await // Materializes all rows in memory
}
```

Figure 22: Unbounded pagination parameters passed to SQL

4. Conclusions

The tested application demonstrates a good security posture, largely thanks to its technology stack. The use of Rust ensures memory safety, while libraries like sqlx and the Vue.js frontend framework effectively mitigate SQLi and XSS. The application prevents common attacks and based on my tests is only flawed by the issues with soft-deleted users, as an attacker can effectively hard-remove an account, given the username or email is known. As username is public, this attack vector is easily exploitable.